Hashing meta::info

Document #:	P3816R2 [Latest] [Status]
Date:	2026-02-23
Project:	Programming Language C++
Audience:	SG7 Reflection

Reply-to:	Matt Cummins <mcummins16@bloomberg.net> Valentyn Yukhymenko <vyuhimenko@bloomberg.net>

Contents

§ 1 Revision history

§ 1.1 Changes from R1:

— Merged the “Ordered maps and sets” section into the “Motivation” section.

— Added annotations example.

§ 1.2 Changes from R0:

— Added a section discussing the different possible semantics of the hash and the reasons for the current choice of behavior.

— Added a section discussing potential ABI concerns.

— Added a section discussing potential extensions to initial design and support.

— Added some more motivation.

§ 2 Abstract

This paper proposes a new standard library template, `consteval_hash<T>`, with a single specialization for `meta::info`. The purpose of this facility is to provide a standard interface for compile-time hashing, thereby allowing unordered containers such as `unordered_map` and `unordered_set` to be used with `meta::info` keys, and potentially with other types in future.

```cpp
consteval auto compile_time_function() -> void
{
    const auto hasher = consteval_hash<meta::info>{}; // proposed
    const size_t h = hasher(^^::);

    // now possible
    unordered_map<meta::info, int, consteval_hash<meta::info>> m;
    unordered_set<meta::info, consteval_hash<meta::info>>      s;
}
```

§ 3 Motivation

[P2996] introduces `meta::info` to represent reflections of C++ constructs. Hashing support was intentionally omitted, as it lies outside the core reflection feature.

[P3372] makes unordered associative containers `constexpr`. This creates a usage for compile-time hashing, including support for `meta::info` and other consteval-only types as keys.

A robust hash for `meta::info` requires compiler support, thus we believe such a facility belongs in the standard library.

Given that this proposal focuses on enabling unordered associative containers keyed on `meta::info`, it is natural to also ask what meaning, if any, should be assigned to `map<meta::info, T>` and `set<meta::info>`. Both of these would require `less<meta::info>` to be well-defined, which would require `operator<` to be defined. However, reflections do not admit a natural or semantically meaningful total ordering. In addition to that, [P2830] (4.1.4) states that any `operator<=>` defined for `std::meta::info` should be consistent with compile-time type ordering it proposes.

Ultimately, defining such an ordering introduces additional semantic challenges and design complexity, so this proposal intentionally restricts its scope to hashing and leaves ordering of reflections for future work.

Enabling hash-based associative containers as a first step will improve the ergonomics of compile-time programming and bring it more in line with runtime code.

In terms of design, a straightforward approach would be to specialize `hash<meta::info>`, but `hash<T>` in general is not constexpr-friendly due to its runtime requirements, so it would be inconsistent with compile-time hashing of other types. As such, this paper proposes a dedicated facility for compile-time hashing which can be extended to other types in the future.

§ 4 Examples

The following examples illustrate practical applications of `consteval_hash`.

§ 4.1 Mp11 `mp_unique` using reflection

In [P2830] (7.3) the authors discuss the infeasibility of implementing `mp_unique` using value-based reflection. Our proposal provides a short and effective solution without needing to sort a list of reflected types.

```
template <typename... Types>
struct type_list {};

template <typename TypeList>
consteval auto mp_unique_reflected()
{
    static_assert(meta::has_template_arguments(^^TypeList), "mp_unique re-
        quires a type_list");
    static_assert(meta::template_of(^^TypeList) == ^^type_list, "mp_unique
        requires a type_list");

    unordered_set<meta::info, consteval_hash<meta::info>> seen;
    vector<meta::info> unique_types;

    for (auto type_info : meta::template_arguments_of(^^TypeList)) {
          if (const bool is_unique = seen.insert(type_info).second; is_u-
        nique) {
            unique_types.push_back(type_info);
        }
    }

    return meta::substitute(^^type_list, unique_types);
}

template <class TypeList>
using mp_unique = [:mp_unique_reflected<TypeList>():];

using input = type_list<int, char, int, string, double, char>;
using filtered = mp_unique<input>;
using expected = type_list<int, char, string, double>;

static_assert(is_same_v<expected, filtered>);
```

§ 4.2 Custom `name_of` function

In [P2996] (4.4.6), the authors discuss the challenges of producing user-friendly names for reflected entities and argue that functions such as `name_of` should be implemented by third-party C++ libraries rather than standardized, with the standard providing the necessary lower level tools to do so. To implement such a function, one might define a custom mapping of known types or entities to more descriptive labels.

Using `unordered_map` for the mapping provides a terser implementation:

Without map

```
consteval name_of(meta::info r) -
        > string_view
{
    if (r == ^^int) {
        return "integer";
    }
    if (r == ^^float) {
        return "32-bit float";
    }
    if (r == ^^double) {
        return "64-bit float";
    }
    if (r == ^^bool) {
        return "boolean";
    }
    if (r == ^^unsigned) {
        return "unsigned
        integer";
    }
    if (r == ^^::) {
        return "global
        namespace";
    }
    if (r == ^^MyType) {
        return "my library type";
    }
    // add more as required

    // Fall back to identifier if
        it exists
    if (meta::has_identifier(r))
        {
        return
        meta::identifier_of(r);
    }
    return "<unnamed>";
}
```

With map

```
consteval name_of(meta::info r) ->
        string_view
{
    unordered_map<meta::info,
        string_view,
        consteval_hash<meta::info>>
        names = {
        {^^int,      "integer"},
        {^^float,    "32-bit
        float"},
        {^^double,   "64-bit
        float"},
        {^^bool,     "boolean"},
        {^^unsigned, "unsigned
        integer"},
        {^^::,       "global
        namespace"},
        {^^MyType,   "my library
        type"},
        // add more as required
    };

    if (auto it = names.find(r);
        it != names.end()) {
        return it->second;
    }

    // Fall back to identifier if
        it exists
    if (meta::has_identifier(r)) {
        return
        meta::identifier_of(r);
    }
    return "<unnamed>";
}
```

§ 4.3 Annotations by type

[P3394] introduces annotations for reflection and provides two query functions:

```
consteval vector<info> annotations_of(info item);
consteval vector<info> annotations_of_with_type(info item, info type);
```

The second function is provided because grouping annotations by their type is a common operation. If a standard compile-time hashing facility was available, then for cases that require calling this function several times with different types, it might be more ergonomic to have an additional function:

```
consteval unordered_map<info, vector<info>, consteval_hash<info>> annota-
        tions_of_by_type(info item);
```

§ 5 Impact

§ 5.1 On the Standard

This proposal is a pure library addition and does not depend on any other library extensions. It introduces the first example of a compile-time hash facility. Moreover, it enables future standard library features: new `consteval` functions may naturally require unordered containers as part of their interfaces, and `consteval_hash` provides the uniform mechanism needed to support such APIs.

§ 5.2 On existing code

As this is a new type in the `std` namespace, this change does not break any existing code, except in the unlikely event that a user has added their own `consteval_hash` type to `std`, which is already undefined behavior.

§ 5.3 On ABI

This proposal does raise some concerns regarding ABI. The issue arises from the fact that we do not require hash values to remain consistent across different translation units. Consider the following simple example:

```
// library.hpp
#include <meta>

constexpr   auto   buffer_size   =   std::consteval_hash<std::meta::info>{}
        (^^int) & 0xFFFF;

struct buffer
{
    char data[buffer_size];
};

buffer get_important_data();
```

```
// library.cpp
#include "library.hpp"
#include <print>

buffer get_important_data()
{
    auto buf = buffer{};
    std::print("size of buffer in library: {}\n", sizeof(buf));
    return buf;
}
```

```
// main.cpp
#include "library.hpp"
#include <print>

int main()
{
    const auto buf = get_important_data();
    std::print("size of buffer in main: {}\n", sizeof(buf));
};
```

Here, the buffer size depends on the hash of `^^int`. Using an implementation where the hash values are different between translation units, this program produces:

```
size of buffer in library: 25040
size of buffer in main: 40384
```

One might think we could require hash values to be stable across translation units, but this would only address static builds. To ensure correctness in all builds, compiler vendors would need to guarantee that all hash values for all reflections never change, which we believe is too restrictive. The same concern applies to any other standard specialization of `consteval_hash<T>`. Furthermore, it may be undesirable for hash values to remain stable in the first place; see the next section for more discussion.

Granted, the cases where ABI issues arise are somewhat contrived and require using `consteval_hash` in a non-typical way. We believe this is acceptable as the current reflection capabilities already make it easy to introduce such an ABI problem. For instance, consider the above example again, but with the buffer size given by

```
constexpr auto buffer_size = std::meta::members_of(^^::, std::meta::access_context::current()).size();
```

Now the size of the buffer depends on the number of members in the global namespace per translation unit, and has exactly the same problem.

§ 5.4 On undefined behavior

Aside from ABI problems, this proposal applies only to compile-time programming, where undefined behavior is disallowed. Accordingly, it does not introduce additional undefined behavior into the standard.

§ 6 Design decisions

§ 6.1 The semantics

For the specific hash value of a given object, we make no specification on what the value should be other than that the likeliness of a collision should be very low, as is expected from a decent hash. Further, hashing the same object multiple times (within a translation unit) will return the same value each time for any given compiler run.

Something that requires more discussion is the stability of the returned values across different compilations, as there are pros and cons for different types which we discuss below.

§ 6.1.1 Unstable hash

This would produce a different hash value for a given object each time the compiler is run.

— Pro: it is trivial to implement for `meta::info` as you can simply hash the pointer into the AST that the reflection represents.

— Pro: the randomness makes it harder for attackers to exploit knowledge of the hash values and precompute collisions.

— Con: it adds randomness into the compiler that cannot be controlled by the user (hindering reproducability). It would then be possible for repeated builds (such as a nightly job) to sporadically fail.

§ 6.1.2 Stable hash

For a stable hash, the values produced stay the same across repeat compiler runs.

— Pro: having a stable hash allows users to implement their own unstable hash provided they have a compile-time source of randomness.

— Con: this is significantly harder to implement for `meta::info`; it essentially requires looking at the value of the reflection and finding something to hash there, e.g. a name. Additionally, this approach naturally also makes the hash values stable across translation units.

— Con: it has more of a maintenance cost; if a future standard introduces a new C++ construct that we can reflect on, then the hash implementation will need to be updated to handle it.

§ 6.1.3 Semi-stable hash

We attempted to find a middle ground that captures the best of both worlds. One such compromise is to ensure stability of values when recompiling the same source code, while still allowing those values to change when the source code itself changes. This preserves the benefits of an unstable hash while enabling reproducible builds, and is currently what this paper proposes.

§ 6.2 The API

§ 6.2.1 `hash<meta::info>`

Ultimately, the goal of this paper is to provide a robust way to hash values of type `meta::info`. The most "obvious" way to do this is to implement `hash<meta::info>`, however this introduces inconsistencies:

— Because of the runtime requirements on `hash<T>`, existing specializations cannot be made `constexpr`, meaning we would end up with some specializations of `std::hash<T>` being consteval-only, while others are runtime-only.

— This then makes using the unordered containers at compile-time inconsistent; depending on the key type, you would sometimes be able to use the default hash, while with others you would need to use another.

— When looking at some code involving the unordered containers, readers would not be able to tell just from the code whether the hash is runtime-only or compile-time-only, and would have to look back at the definition of hash. Having a distinct type so that hash is always runtime-only solves this.

§ 6.2.2 `hash_of`

We could sidestep the issues associated with `hash<meta::info>` by instead providing a free function:

```
namespace std::meta {
    auto hash_of(info r) -> size_t;
}
```

This would be defined in `<meta>`. If users need to use `meta::info` as keys in compile-time hash maps, they can use it to implement their own hash (like they will have to do with all other types currently). However, we do not propose this because:

— The functions in `<meta>` provide fundamental details about reflections, and having a `hash_of` function suggests there is a single meaningful way of hashing which is obviously not true. `hash<T>` solves this by simply being understood to provide *a* reasonable default implementation, not *the* implementation.

  — The caveat here is that this is not quite true; `<meta>` also provides `display_string_of`, which provides some reasonable string representation of the given reflection. But aside from this single function, the point still holds.

— More crucially, if we provide users with functionality that they need to wrap themselves, we should just standardize the boilerplate instead. Which leads to…

§ 6.2.3 `consteval_hash<meta::info>`

This brings us to the proposal in this paper. It has a few benefits:

— It is similar to `hash_of`, but does not have the same drawback of suggesting that it is the single meaningful way to hash `meta::info`. It carries the same semantics of a "reasonable implementation" that `hash<T>` has.

— Users won't need to write their own wrapper of `hash_of`.

— Even if `hash<T>` could be made constexpr for other types (which it can't), there would still be an issue with value consistency between runtime and compile-time for pointers, and possibly other types. Separating runtime and compile-time hashing into

two separate types avoids this issue, as you shouldn't expect two hashing algorithms to give the same result.

  - Because of this, with a new type you could easily provide `consteval_hash<T*>` as well.

- It can be easily extended for all other standard and builtin types that have a specialization of hash, making compile-time map usage far more uniform.

It also has a few obvious downsides:

- Whenever a new specialization of `hash<T>` is standardized, it likely will also want a corresponding `consteval_hash<T>`, increasing the burden on implementers.
- When users implement `hash<T>` for their own type, they may not care about hash salting and just implement their `hash<T>::operator()` as `constexpr`, which is more natural than also implementing `consteval_hash` and easier to use. However, this is unlikely to be a significant concern for the standard.
- The use of unordered containers in `constexpr` functions remains awkward. Different types are required at compile-time versus runtime, often necessitating heavy use of `if consteval`. However, this is already a problem, and this proposal is not intended to address it.

Overall, this feels like the more complete and extensible solution, so it is the one we are proposing.

§ 6.2.4 Alternate names for `consteval_hash`

There are a few other names we considered. Below is a list of them as well as the reasons we decided against them.

Name	Comments
`stable_hash<T>`	This name would be better suited to a `constexpr` hash usable at both runtime and compile-time. To us, this name does not capture the core feature of the proposed hash, that is to be compile-time only. Our specification for the new type also does not guarantee stability across translation units, so this name is misleading.
`static_hash<T>`	The keyword static in C++ already means "compile-time" in some cases, e.g. `static_assert`, however the keyword is overloaded with many other meanings, so could be confusing. consteval is the keyword for compile-time, hence `consteval_hash`.
`meta_hash<T>`	Concise and clearly related to compile-time functionality. However, the word "meta" more closely relates to reflection, which is a subset of

	compile-time functionality, and one which compile-time hashing does not necessarily have anything to do with.
`fixed_hash<T>`	Similar to `stable_hash<T>` in meaning.
`compile_time_hash<T>`	This name best describes what it does, but given that C++ already has the `consteval` keyword to mean "compile-time-only", this name is less consistent with the rest of the language.
`comptime_hash<T>`	Although we love Zig, we are proposing a C++ feature!
`ct_hash<T>`	Terse, but too terse. Would you guess that it was a shortening of "compile-time hash" at first glance?
`ce_hash<T>`	Same as above. Would you guess it was a shortening of "consteval hash"?
`compile_time_only_hash<T>`	Consistent with `move_only_function`, but far too long.
`consteval_only_hash<T>`	A slight improvement on the above, but the "only" is superfluous since consteval already denotes that it only works at compile-time.
`constexpr_hash<T>`	Just incorrect, implies that it is usable at runtime too. If such a type existed, you would expect its interface to be made up of `constexpr` functions, not `consteval`.
`hash<T, hashtype::compile_time>`	Rather than a new type, we could instead extend `hash<T>` by providing an enum class to select what kind of hash you want. This enum could be extended to provide even more hashes in the future. This feels far messier, and given that it would be impossible to implement certain hashes for certain types, it would be misleading and provide an API that looks incomplete.

§ 7 Proposed wording

The proposed wording below is for a *semi-stable* implementation.

§ 7.1 The specification for `consteval_hash<T>`

Add a new section, `ConstevalHash` [constevalhash.requirements], defined analogously to `Cpp17Hash` [hash.requirements]:

> A type `H` meets the `ConstevalHash` requirements if
>
> 1 — It is a function object type ([function.objects]).

2 — It meets the `Cpp17CopyConstructible` and `Cpp17Destructible` requirements.

3 — It is a consteval-only type ([basic.types.general]).

4 — Given two instances of H, it is not guaranteed that they will produce the same values for the same arguments.

(4.1) — [*Note:* In particular, the values may be different between translation units. — *end note*]

5 — The expressions in the table below are valid and have the indicated semantics:

Given `Key` is an argument type for function objects of type `H`, `h` is a value of type (possibly `const`) `H`, `u` is an lvalue of type `Key`, and `k` is a value of type convertible to (possibly `const`) `Key`.

Expression	Return type	Requirement
`h(k)`	`size_t`	The value returned shall depend only on `k`. The value shall be stable across repeated compiler runs. [*Note:* Modifying the source code may change the value. — *end note*] For two different values `t1` and `t2`, the probability that `h(t1)` and `h(t2)` compare equal should be very small, approaching `1.0 / numeric_limits<size_t>::max()`.
`h(u)`	`size_t`	Shall not modify `u`.

Add a new section, "Class template consteval_hash" [unord.consteval_hash], defined analogously to "Class template hash" [unord.hash]. The only difference is that this currently makes no mention of specializations for `nullptr_t` or for cv-unqualified arithmetic, enumeration and pointer types (which can be added later):

Class `template consteval_hash`

1 — Each specialization of `consteval_hash` is either enabled or disabled, as described below.

(1.1) — [*Note:* Enabled specializations meet the `ConstevalHash` requirements, and disabled specializations do not. — *end note*]

2 — If the library provides an explicit or partial specialization of `consteval_hash<Key>`, that specialization is enabled except as noted otherwise, and its member functions are `noexcept` except as noted otherwise.

3 — If `H` is a disabled specialization of `consteval_hash`, these values are `false`: `is_default_constructible_v<H>`, `is_copy_constructible_v<H>`, `is_copy_assignable_v<H>`, and `is_move_assignable_v<H>`. Disabled specializations of `consteval_hash>` {.cpp} are not function object types ([function.objects]).

(3.1) — [*Note:* This means that the specialization of consteval_hash exists, but any attempts to use it as a ConstevalHash will be ill-formed. — *end note*]

4 — An enabled specialization `consteval_hash<Key>` will:

(4.1) — Meet the `ConstevalHash` requirements, with `Key` as the function call argument type, the `Cpp17DefaultConstructible` requirements, the `Cpp17CopyAssignable` requirements, the `Cpp17Swappable` requirements.

(4.2) — Meet the requirement that if `k1 == k2` is `true`, `h(k1) == h(k2)` is also `true`, where `h` is an object of type `consteval_hash<Key>` and `k1` and `k2` are objects of type Key.

(4.3) — Meet the requirement that the expression `h(k)`, where `h` is an object of type `consteval_hash<Key>` and `k` is an object of type `Key`, shall not throw an exception unless `consteval_hash<Key>` is a program-defined specialization.

§ 7.2 The specialization for `meta::info`

Add a new section [meta.reflection.hash]:

```
template <typename T> struct consteval_hash;
template <> struct consteval_hash<meta::info>;
```

The specialization is enabled ([unord.consteval_hash]).

§ 7.3 Feature testing macros

Add two new feature macros into [version.syn], one for the new type, and one for the `meta::info` instantiation:

```
#define __lib_consteval_hash_template YYYYXXL  // also in <meta>
#define __lib_consteval_hash_meta_info YYYYXXL // also in <meta>
```

§ 8 Implementation experience

We have implemented an unstable version and two semi-stable versions of the hash on Bloomberg's Clang fork.

The first semi-stable approach is more robust and relies on hashing certain properties of the underlying reflected construct. For example, for a namespace, we hash the source loca-

tion. We are currently working on modifying this to create a stable hash. This can be done by relying on more stable properties of the reflected constructs; for example, using the name of a namespace rather than the source location.

The second semi-stable approach is to take an otherwise unstable hash implementation and add a layer of interning: maintain a map from AST nodes (keyed by their pointer addresses) to integer IDs, assigning each ID from an incrementing counter. This yields a straightforward, semi-stable hash implementation. It also avoids the need for updates when new reflectable constructs are added to the language, since the mechanism depends solely on pointer identity rather than on the structure of the reflected entities.

Like with the rest of the reflection API, `consteval_hash<meta::info>::operator()` can be implemented via a compiler intrinsic.

```
template<>
struct consteval_hash<meta::info>
{
  consteval consteval_hash() = default;
  consteval consteval_hash(const consteval_hash<meta::info>&) = default;
  consteval consteval_hash(consteval_hash<meta::info>&&) = default;

  consteval auto operator()(meta::info r) const noexcept -> size_t {
    return __metafunction(meta::detail::__metafn_reflection_hash, r);
  }

private:

  // This unused variable is here to make consteval_hash<> a
  // consteval-only type.
  [[maybe_unused]] const meta::info unused = ^^::;
};
```

[P3068] has been approved for C++26, allowing exception throwing within constexpr, so it is meaningful to mark `consteval_hash<meta::info>::operator()` as `noexcept`.

§ 9 Possible extensions of current design

§ 9.1 More predefined `consteval_hash<T>` specializations

As mentioned before, `consteval_hash<T>` can be easily extended for all other standard and builtin types that have a specialization of `std::hash<T>`.

To improve initial usability and reduce unnecessary boilerplate for common cases, we could extend the scope of the original design by providing a small set of predefined specializations.

Specifically, we are proposing to add predefined `consteval_hash<T>` specializations for:

— all cv-unqualified arithmetic types,

— all cv-unqualified enumeration types,

— all cv-unqualified pointer types, and

— `std::nullptr_t`

These types are both widely used as keys in unordered associative containers and straightforward to support in a portable, implementation-independent manner.

Notably, extending initial support to pointer types would help in resolving the implementation experience issues highlighted in [P3372] concerning `std::hash<T*>` having inconsistent values between compile time and runtime. By separating compile-time hashing into a distinct type, this inconsistency is no longer an problem.

Adding support to other library types, in order to achieve feature parity with `std::hash<T>`, should be discussed in separate proposals, as not all currently supported types are available at compile time and some may require additional limitations.

§ 9.2 Opt-in support of randomized hashing

By having hash values stable across repeated compilation runs, we dismiss some use-cases when randomness is desirable, e.g. compile-time obfuscation.

A natural extension of this feature is therefore to provide an *opt-in* mechanism that enables randomized hashing at compile time.

This preserves stability as the default behavior while supporting those specific scenarios that benefit from enabling randomness.

One straightforward implementation strategy is to introduce a compile-time template parameter controlling whether randomness is enabled:

```
enum class hash_semantics {
    stable,
    unstable
};

template<typename T, hash_semantics S = hash_semantics::stable>
struct consteval_hash;

template<hash_semantics S>
struct consteval_hash<meta::info, S>;

consteval_hash<meta::info>           deterministic;   // stable across re-
        peated runs
consteval_hash<meta::info,  hash_semantics::unstable>          randomized;
        // random for every compilation
```

§ 10 Acknowledgements

— Dan Katz for the original implementation and wording, as well as his work on implementing the main reflection paper in Clang.

— Hana Dusíková for allowing unordered containers to be usable in constexpr, which is a primary motivator for this paper.

§ 11 References

[P2830] Nate Nichols and Gašper Ažman. Constexpr Type Ordering.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2830r10.html

[P2996] Wyatt Childers, Peter Dimov, Dan Katz, Barry Revzin, Andrew Sutton, Faisal Vali, and Daveed Vandevoorde. Reflection for C++.
https://isocpp.org/files/papers/P2996R13.html

[P3068] Hana Dusíková. Allowing Exception Throwing in Constant-Evaluation.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3068r2.html

[P3372] Hana Dusíková. Constexpr Containers and Adaptors.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3372r3.html

[P3394] Wyatt Childers, Dan Katz, Barry Revzin, and Daveed Vandevoorde. Annotations for Reflection.
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3394r4.html